Sieci Neuronowe
Laboratorium 03
Budowanie wielowarstwowej w pełni połączonej sieci neuronowej

Andrzej Kordecki

Zakład Teorii Maszyn i Robotów
Instytut Techniki Lotniczej i Mechaniki Stosowanej
Wydział Mechaniczny Energetyki i Lotnictwa
Politechnika Warszawska
2026

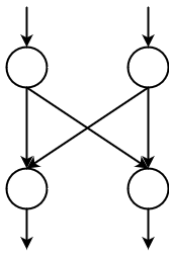
Wstęp

Celem laboratorium jest opracowanie w języku Python, z wykorzystaniem biblioteki NumPy, implementacji wielowarstwowej sieci neuronowej. Ćwiczenie stanowi kontynuację poprzednich zajęć, podczas których przygotowano podstawowe funkcje aktywacji i funkcje strat. Obecnie skupimy się na budowie struktury sieci i propagacji sygnału w przód (ang. *forward propagation*), czyli obliczaniu jej wyjścia dla zadanych danych wejściowych i wag. Uczenie sieci, w tym obliczanie gradientów i aktualizacja wag, będzie implementowana na kolejnych laboratoriach.

Opracowany program będzie rozwijany w następnych ćwiczeniach, dlatego istotne jest zachowanie spójnej struktury danych oraz rozdzielenie definicji architektury sieci od jej parametrów i wyników obliczeń.

Sieć neuronowa

Sieć neuronowa jest strukturą złożoną z połączonych neuronów, które wykonują operacje na sygnałach wejściowych. W laboratorium będziemy rozpatrywać sieci jednokierunkowe (ang. *feedforward neural networks*), w których przepływ sygnału odbywa się od wejścia do wyjścia, bez połączeń zwrotnych (rys. 1).

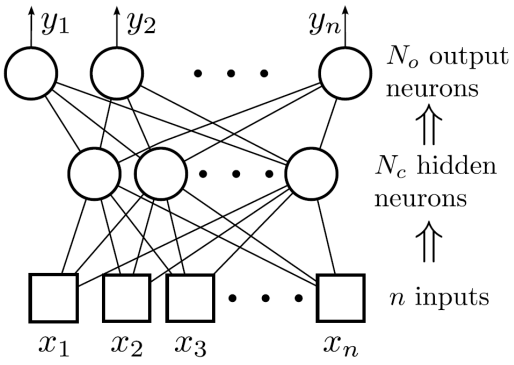


Rysunek 1: Przykład sieci jednokierunkowej.

Sieci te charakteryzują się następującymi właściwościami:

- informacja jest przekazywana w określonym kierunku, bez cykli w grafie połączeń;
- nie wszystkie neurony muszą być połączone – w tym laboratorium ograniczymy się jednak do warstw w pełni połączonych (ang. *dense*);
- wynik sieci jest obliczany poprzez kolejne przekształcenia sygnałów przez warstwy. Dla trzech warstw obliczeniowych można go schematycznie zapisać jako $F(X) = F_3(F_2(F_1(X)))$.

Wielowarstwowa sieć neuronowa, którą będziemy implementować, składa się z warstwy wejściowej, co najmniej jednej warstwy ukrytej i warstwy wyjściowej (rys. 2). Warstwa wejściowa określa rozmiar danych wejściowych; w przyjętej implementacji nie wykonuje mnożenia przez wagi ani funkcji aktywacji. Operacje te wykonywane są w warstwach typu `dense`. Wyjściem programu będzie wynik propagacji sygnału przez wszystkie kolejne warstwy.



Rysunek 2: Przykład wielowarstwowej sieci jednokierunkowej z jedną warstwą ukrytą.

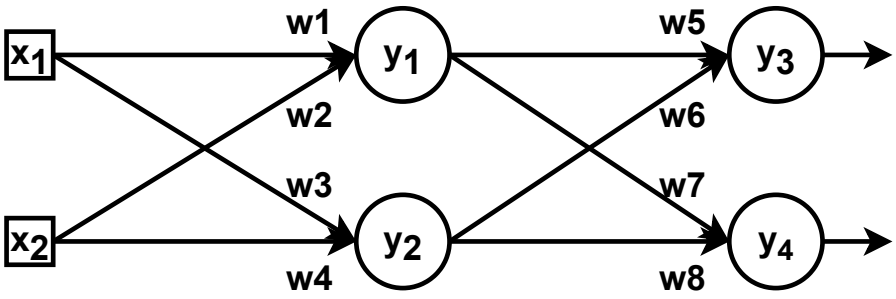
Propagacja sygnału w przód i obliczenia macierzowe

Wyjście wyjściowe sieci można wyznaczać wykonując po sobie i kolejno obliczenia osobno dla każdego neuronu, obliczając jego potencjał aktywacji i następnie wartość funkcji aktywacji. Takie podejście jest jednak mniej wygodne i zwykle mniej wydajne niż jednoczesne obliczenia dla całej warstwy sieci.

Wykorzystanie macierzy wag przypisanej warstwie pozwala zastąpić wiele operacji na pojedynczych neuronach jednym mnożeniem macierzowym, które biblioteka NumPy wykonuje za pomocą zoptymalizowanych procedur. Nie zmniejsza to liczby matematycznych mnożeń i dodawań wymaganych przez daną warstwę, ale eliminuje konieczność wykonywania ich w pętli Pythona i ułatwia przetwarzanie wielu próbek równocześnie. Dla uproszczenia, zgodnie z założeniami wcześniejszego laboratorium, na tym etapie pomijamy obciążenie (bias). W dalszej części zakładamy również, że wszystkie neurony danej warstwy używają tej samej funkcji aktywacji.

Przykład: dwie warstwy obliczeniowe

Rozważmy sieć o dwóch wejściach, dwóch neuronach pierwszej warstwy obliczeniowej i dwóch neuronach warstwy wyjściowej (rys. 3). Potencjały i wyjścia neuronów pierwszej warstwy mają postać:



Rysunek 3: Sieć z dwiema warstwami obliczeniowymi.

$$v_1 = w_1 x_1 + w_2 x_2, \qquad y_1 = f_1(v_1),$$
$$v_2 = w_3 x_1 + w_4 x_2, \qquad y_2 = f_2(v_2).$$

Dla drugiej warstwy wejściami są wartości y_1 oraz y_2:

$$v_3 = w_5 y_1 + w_6 y_2, \qquad y_3 = f_3(v_3),$$
$$v_4 = w_7 y_1 + w_8 y_2, \qquad y_4 = f_4(v_4).$$

Zapiszmy wejście i macierze wag w postaci:

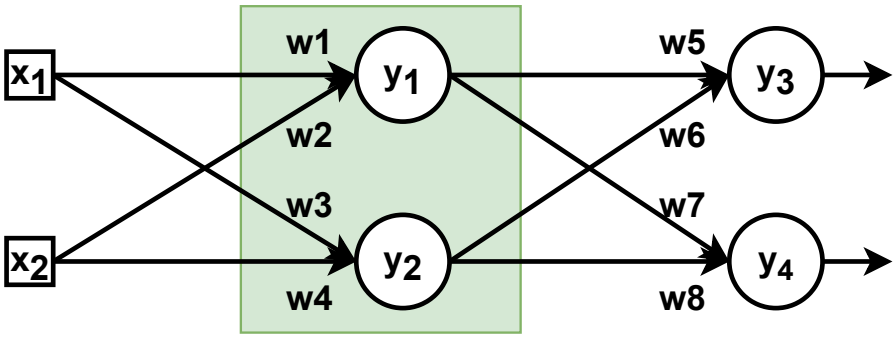
$$X = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \qquad W_1 = \begin{bmatrix} w_1 & w_2 \\ w_3 & w_4 \end{bmatrix}, \qquad W_2 = \begin{bmatrix} w_5 & w_6 \\ w_7 & w_8 \end{bmatrix}.$$

Wiersz macierzy wag odpowiada jednemu neuronowi warstwy, a kolumna jednemu wejściu. Implementacja wartości wstępnych macierzy wag w programie powinna wykorzystywać wartości losowe `np.random.randn(units, previous_units)`.

Potencjały pierwszej warstwy obliczamy jednocześnie (rys. 4):

$$V_1 = \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = W_1 X = \begin{bmatrix} w_1 & w_2 \\ w_3 & w_4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}.$$

Neurony w pierwszej warstwie wykorzystują tę samą funkcję aktywacji $f^{(1)}$:

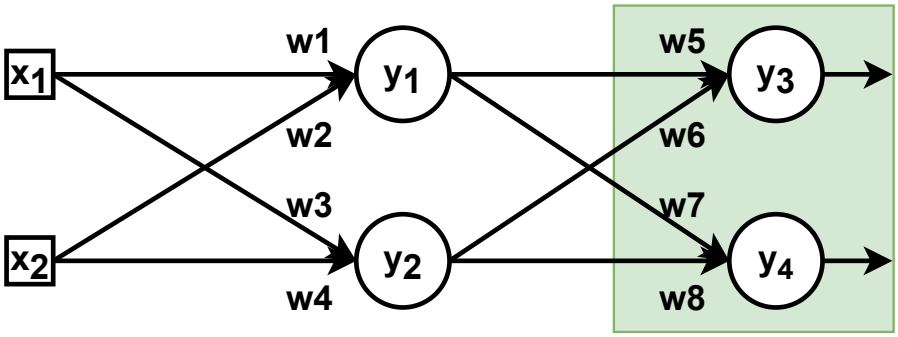


Rysunek 4: Obliczenia potencjałów pierwszej warstwy.

$$Y_1 = F_1(V_1) = \begin{bmatrix} f^{(1)}(v_1) \\ f^{(1)}(v_2) \end{bmatrix} = F_1(W_1 X).$$

Analogicznie obliczamy wyjście dla drugiej warstwy (rys. 5) sieci:

$$V_2 = W_2 Y_1, \qquad Y_2 = F_2(V_2) = F_2(W_2 Y_1).$$



Rysunek 5: Obliczenia w drugiej, wyjściowej warstwie.

Dla sieci mającej L warstw z wagami zapis uogólnia się do:

$$\boxed{V_l = W_l Y_{l-1}, \qquad Y_l = F_l(V_l), \qquad Y_0 = X, \quad l = 1, \ldots, L.}$$

Gdy warstwa $l-1$ ma n_{l-1} wyjść, a warstwa l ma n_l neuronów, macierz W_l ma rozmiar $n_l \times n_{l-1}$. Dla pojedynczej próbki Y_{l-1} ma rozmiar $n_{l-1} \times 1$, a Y_l ma rozmiar $n_l \times 1$.

Wiele próbek jednocześnie. Przy tej samej konwencji każda *kolumna* macierzy X odpowiada jednej próbce. Jeśli próbki są ułożone w macierzy $X \in \mathbb{R}^{n_0 \times m}$, gdzie m oznacza ich liczbę, wzór $V_l = W_l Y_{l-1}$ pozostaje bez zmian, a $Y_l \in \mathbb{R}^{n_l \times m}$. Taką organizację danych można uzyskać także z listy, której elementy przechowują wektory poszczególnych wejść (cech) dla wszystkich próbek, zgodnie z koncepcją poprzedniego laboratorium.

Struktura sieci neuronowej w kodzie programu

Celem ćwiczenia jest implementacja sieci **w pełni połączonej**, której architekturę można dowolnie określać za pomocą listy. Kolejne elementy listy są słownikami opisującymi kolejne warstwy sieci. Przyjmujemy następujące pola:

- `type` – typ warstwy: `'input'` lub `'dense'`;
- `units` – liczba wejść w warstwie wejściowej albo liczba neuronów w warstwie `dense`;
- `activation_function` – nazwa funkcji aktywacji warstwy `dense`, zgodna z nazwami z poprzedniego laboratorium.

Przykładowa definicja sieci z jednym wejściem, ośmioma neuronami ukrytymi i dwoma neuronami wyjściowymi:

```
structure = [
{'type': 'input', 'units': 1},
{'type': 'dense', 'units': 8, 'activation_function': 'tanh'},
{'type': 'dense', 'units': 2, 'activation_function': 'linear'}
]
```

Sama definicja architektury nie zawiera macierzy wag ani wartości pośrednich potrzebnych do propagacji. Dlatego przygotowujemy funkcję, która tworzy *roboczą reprezentację sieci* i dla każdej warstwy `dense` przechowuje:

- `weights` – macierz wag;
- `activation_function` – wybraną funkcję aktywacji;
- `activation_potential` – obliczone potencjały neuronów;
- `output` – wartości wyjściowe warstwy;
- `delta` – miejsce na wielkości obliczane podczas propagacji wstecznej w następnych laboratoriach.

Poniżej znajduje się przykład funkcji tworzącej taką strukturę. Wykorzystujemy ten sam sposób inicjalizacji wag jak w poprzednim laboratorium; dalsze modyfikacje inicjalizacji będą możliwe później.

```
import numpy as np

def create_network(structure):
  network = [structure[0].copy()]
  network[0]['output'] = None

  for i in range(1, len(structure)):
    layer = {
      'type': structure[i]['type'], # pomijana pozniej ze wzgledu na jeden typ warstw
      'units': structure[i]['units'], # pomijana pozniej ze wzgledu na wyznczenie wag
      'weights': np.random.randn(structure[i]['units'], structure[i - 1]['units']),
      'activation_function': structure[i]['activation_function'],
      'activation_potential': None,
      'output': None,
      'delta': None
    }
    network.append(layer)

  return network
```

Przykład utworzenia sieci z dwiema warstwami ukrytymi:

```
structure = [
{'type': 'input', 'units': 1},
{'type': 'dense', 'units': 8, 'activation_function': 'tanh'},
{'type': 'dense', 'units': 8, 'activation_function': 'relu'},
{'type': 'dense', 'units': 1, 'activation_function': 'linear'}
]
network = create_network(structure)
```

W tym laboratorium pole `delta` nie będzie jeszcze wykorzystywane. Zostaje przewidziane na potrzeby przyszłej implementacji uczenia metodą propagacji wstecznej. Pole `activation_potential` pozostaje odrębne od `output`, ponieważ w przyszłości będzie potrzebne także do obliczania pochodnych funkcji aktywacji.

Ważne: Przykład pokazuje proponowaną strukturę danych, a nie kompletny algorytm propagacji. Studenci mogą samodzielnie zmodyfikować organizację kodu, pod warunkiem zachowania wymaganej funkcjonalności i możliwości dalszego rozwijania programu w kolejnych laboratoriach.

Zadanie 1

Uzupełnij klasy funkcji aktywacji o brakujące metody zgodnie z zapisem programu poniżej. Brakujące metody to: tangens hiperboliczny, funkcja logistyczna i ReLu. Obecnie funkcje aktywacji nie muszą wyznaczać pochodnych. Przy pisaniu tego punktu można wykorzystać kod napisany podczas poprzedniego laboratorium.

```
import sys

class Activation_fcn:
  def __init__(self):
    self.functions = {
      'linear': self.linear,
      'sigmoid': self.logistic,
      'logistic': self.logistic,
      'tanh': self.tanh,
      'relu': self.relu
    }

  def output(self, layer, name, derivative=False):
    if name in self.functions:
      return self.functions[name](layer['activation_potential'], layer['output'], derivative)
    else:
      sys.exit(f"Error: Activation function '{name}' not found.")

  # Identity
  def linear(self, activation_potential, output, derivative=False):
    pass # do uzupelnienia

  # Logistic (sigmoid)
  def logistic(self, activation_potential, output, derivative=False):
    pass # do uzupelnienia

  # Hyperbolic tangent
  def tanh(self, activation_potential, output, derivative=False):
    pass # do uzupelnienia

  #  ReLU
  def relu(self, activation_potential, output, derivative=False):
    pass # do uzupelnienia
```

Zadanie 2

Stwórz klasę "Neural_network"do tworzenia i operacji na sieciach neuronowych. Klasa powinna zawierać funkcję "create_network", która zbuduje strukturę sieci neuronowej. Istotnym parametrem funkcji będzie nowy argument funkcji "init_weight", który określi metodę inicjalizacji wag sieci neuronowej:

- 'zero' - wszystkie wartości wag będą miały wartość 0,
- 'one' - wszystkie wartości wag będą miały wartość 1,
- 'rand' - wszystkie wartości wag będą miały wartość losową (kod poniżej),

Zaimplementuje nową funkcjonalność klasy umożliwiającą generowanie początkowych wag o określonej wartości. Ustawienie wszystkich wag na taką samą wartość będzie tylko wykorzystywane do sprawdzenia poprawności implementacji sieci. Poniżej zwarto kod programu, który wymaga modyfikacji.

```
class Neural_network(object):

    def __init__(self, structure=None, init_weight='rand'):
        self.af = Activation_fcn()
        if structure:
            self.create_network(structure, init_weight='rand')

    def create_network(self, structure, init_weight='rand'):
        self.nnetwork = [structure[0]]
        for i in range(1, len(structure)):
            new_layer = {
                # Tu nalezy wykonac odpowiedniej modyfikacje
                'weights': np.random.randn(structure[i]['units'], structure[i-1]['units']),
```

```
                'activation_function': structure[i]['activation_function'],
                'activation_potential': None,
                'delta': None,
                'output': None}
            self.nnetwork.append(new_layer)
        return self.nnetwork

structure = [{'type': 'input', 'units': 1},
             {'type': 'dense', 'units': 8, 'activation_function': 'tanh'},
             {'type': 'dense', 'units': 8, 'activation_function': 'tanh'},
             {'type': 'dense', 'units': 1, 'activation_function': 'linear'}]
model = Neural_network(structure, 'rand')
```

Zadanie 3

Utwórz dwie funkcje "forward_propagate"w ramach klasy "Neural_network"stworzonej w poprzednim zadaniu. Funkcja "forward_propagate"będzie wykonywała propagację sygnału w sieci neuronowej, warstwa po warstwie, od wejście do wyjścia sieci. Wejściem funkcji powinna być jedna próbka danych. Wyjściem funkcji "forward_propagate"będzie wektor wartości wyjściowych ostatniej warstwy sieci neuronowej. Funkcja powinna wyznaczać potencjał neuronu oraz wyjścia neuronów w ramach wewnętrznych obliczeń. Do sprawdzenia działania funkcji należy wykorzystać funkcję "predict", którą należy dodać do klasy "Neural_network". Funkcja "predict"będzie wywoływał funkcje "forward_propagate"dla wszystkich próbek danych i zwróci listę wszystkich wartości estymowanych przez sieć neuronową. Poniżej zamieszczono kod programu, który możemy wykorzystać w rozwiązaniu. Przy pisaniu warto zaznajomić się z funkcjami biblioteki np.matmul(.) oraz np.flatten(.).

```
# Forward propagate input to a network output
def forward_propagate(self, nnetwork, inputs):
    inp = inputs.copy()
    for i in range(1, len(nnetwork)):
        # Tu nalezy wykonac odpowiedniej modyfikacje
        nnetwork[i]['activation_potential'] = None
        nnetwork[i]['output'] = None
        inp = None
    return inp

def predict(self, nnetwork, inputs):
    out = []
    for input in inputs:
        out.append(self.forward_propagate(nnetwork, input))
    return out
```

Zadanie 4

Przetestuj działanie wyznaczonej w poprzednim zdaniu sieci neuronowej na kilku opisanych poniżej ustawieniach i zapisz otrzymane wyniki. Sprawdź przez analizę znanych parametrów sieci, czy otrzymane wyniki są prawidłowe?

1. Dane:

```
structure = [{'type': 'input', 'units': 1},
             {'type': 'dense', 'units': 2, 'activation_function': 'linear'},
             {'type': 'dense', 'units': 1, 'activation_function': 'linear'}]

network = model.create_network(structure, "zero")

n = 1
X = np.linspace(-5, 5, n).reshape(-1, 1)

predicted = model.predict(network, X)
```

2. Dane:

```
structure = [{'type': 'input', 'units': 1},
             {'type': 'dense', 'units': 2, 'activation_function': 'tanh'},
             {'type': 'dense', 'units': 2, 'activation_function': 'tanh'},
             {'type': 'dense', 'units': 1, 'activation_function': 'tanh'}]

network = model.create_network(structure, "one")
```

```
n = 1
X = np.linspace(-5, 5, n).reshape(-1, 1)

predicted = model.predict(network, X)
```

3. Dane:

```
structure = [{'type': 'input', 'units': 1},
             {'type': 'dense', 'units': 4, 'activation_function': 'ReLu'},
             {'type': 'dense', 'units': 4, 'activation_function': 'ReLu'},
             {'type': 'dense', 'units': 1, 'activation_function': 'ReLu'}]

network = model.create_network(structure, "rand")

n = 1
X = np.linspace(-5, 5, n).reshape(-1, 1)

predicted = model.predict(network, X)
```